基于CPU与GPU混合计算的频率分析工具FACG

FACG (Frequency Analysis tool of CPU and GPU mixed computing)是一款为天文时间序列数据设计的、支持 GPU 加速的迭代预白化（Iterative Prewhitening）频域分析工具，其算法核心与数学模型灵感来源于 SigSpec (Reegen 2007, A&A 467, 1353)。

FACG 实现了完整的 SigSpec 数据处理管线——包括频谱显著性估计、迭代预白化以及全局多重正弦非线性优化——同时利用计算后端的 GPU 为计算密集型任务提供深度加速。如果系统中未检测到可用的 GPU，程序将透明、平滑地降级回基于 CPU (NumPy) 的运算模式。

Read this in English 🇬🇧

核心特性

- **GPU 加速支持** (基于 CuPy / PyTorch) — 无缝向后兼容纯 CPU (NumPy) 模式
    - **NVIDIA 显卡**: 通过 CuPy 调用 CUDA 加速
    - **Apple Silicon (M 系列芯片)**: 通过 PyTorch 调用 Metal (MPS) 加速
- **100% 兼容 SigSpec 显著性评估** 公式（基于倾斜概率椭圆的解析假警报概率）
- **迭代预白化** 与基于二分查找法的精确频率锁定
- **全局多重正弦联合优化** 基于 Levenberg–Marquardt 算法，并在 GPU 端实现雅可比（Jacobian）矩阵的解析法提速
- **灵活的文件兼容性** — 支持任意合法文件名作为输入（摒弃了原版复杂的命名格式约束）
- **智能硬件检测** — 自动探测物理显卡硬件，若环境缺失相应依赖 (CuPy/PyTorch) 则发起交互式安装建议，用户确认后可自动降级至 CPU 运行
- **基准测试工具** — 内置 `--testdata` 指令，用于一键生成多组测试数据以进行性能测评
- **配置文件支持** — 支持通过 `--gen-config` 生成 `facg.conf` 以固化常用分析参数
- **易于集成的 Python 包** — 开箱即用的 `facg` 命令行工具以及方便天文流线调用的 Python API
- **性能耗时统计** — 详细报告单次迭代与整体流程的总耗时

项目结构

```
FACG/
├── pyproject.toml               # Python 包构建配置文件
├── README.md                    # 英文说明文档
├── README_CN.md                 # 中文说明文档（本文）
├── FACG.py                      # 最早期的单文件脚本（保留以供参考）
├── Performance_Report.md        # 多平台详细的性能测评报告
└── facg/                        # 核心 Python 源代码包
    ├── __init__.py              # 包入口，导出 FACGConfig 和 run_analysis
    ├── __main__.py              # 命令行入口：facg 或 python -m facg
    ├── backend.py               # GPU/CPU 混合计算后端路由层（CuPy ↔ NumPy）
    ├── config.py                # FACGConfig 数据类 — 定义了所有可调参数
    ├── config_io.py             # .conf 配置文件生成与解析模块
    ├── spectrum.py              # 核心频谱引擎（GPU 批量 DFT、显著性公式、频率精
化）
    ├── optimizer.py             # 多重正弦全局优化器（GPU 雅可比矩阵 + LM）
```

```
        ├── io.py                   # 文件 I/O（支持普通文本、CSV、Excel、FITS）
        └── prewhiten.py            # 迭代预白化级联主循环模块
```

模块分工

模块	职责说明
`backend.py`	自动检测显卡硬件; 封装并导出统一的 `xp` 以及设备传输方法 `to_device()` / `to_host()`，使其他所有核心逻辑无需关心底层后端。
`spectrum.py`	包含 `compute_significance_spectrum()`（GPU 加速的批量 DFT 和 SigSpec 显著性计算）及 `refine_frequency()` 等模块。
`optimizer.py`	包含 `global_optimize()` — 对迄今提取的所有频率、振幅和相位进行 LM 联合非线性拟合。
`prewhiten.py`	`run_analysis()` — 分析主循环：探测峰值 → 精化频率 → 全局优化 → 扣除残差 → 循环。
`config.py`	`FACGConfig` 参数对象配置类。
`config_io.py`	负责将 `facg.conf` 文件参数映射到运行时环境及生成默认配置文档。
`io.py`	允许读入所有格式的空白分隔文本数据，以及按需写出各迭代步骤的频谱、残差、结果等日志文件。
`__main__.py`	基于 `argparse` 的控制台入口程序，接受并解析命令行参数。

安装指南

```
cd FACG
pip install -e .
```

只需一行命令即可完成安装。在程序执行时，FACG 会**自动探测**并匹配最优计算后端：

- 如果系统装有 NVIDIA 显卡并且安装了 CuPy → 启用 GPU CUDA 加速（带终端勾号提示）。
- 如果是 Apple 芯片的 Mac 并安装了 PyTorch → 启用 GPU Metal 加速。
- 如果都不满足 → 透明降级至 CPU 模式，并提供依赖缺失相关的交互式预警：

对于 **Windows/Linux** (含 NVIDIA 显卡):

```
  ✓ CPU (NumPy 1.26.x)
  ⚠ NVIDIA GPU hardware detected, but CuPy is not installed.
    To enable GPU acceleration, install CuPy for your CUDA version:
      pip install cupy-cuda12x      # for CUDA 12.x
      pip install cupy-cuda11x      # for CUDA 11.x
    See https://docs.cupy.dev/en/stable/install.html
```

```
Do you want to continue in CPU mode? [y/N]:
```

对于 **macOS** (含 Apple Silicon 芯片):

```
  ⚠ PyTorch not installed (or no Metal support) — running on CPU only.
    To enable Apple Metal GPU acceleration, install PyTorch:
      pip install torch
```

基础依赖 (pip 会自动安装):

- Python ≥ 3.9
- NumPy ≥ 1.22
- SciPy ≥ 1.8
- Pandas ≥ 1.5
- Openpyxl ≥ 3.0 (解析 Excel 用)
- Astropy ≥ 5.0 (解析 FITS 表格用)
- Matplotlib ≥ 3.5
- *(可选)* CuPy — 开启 NVIDIA CUDA GPU 加速
- *(可选)* PyTorch — 开启 Apple Silicon GPU 加速

快速入门

命令行模式 (CLI)

```
# 分析单个文件（支持任意有效文件名）
facg my_lightcurve.dat

# 批量分析当前目录下所有符合条件的文件
facg file1.dat file2.dat file3.dat

# 附加自定义参数
facg data.dat --sig-limit 6 --oversampling 40 --freq-high 50 --output-dir
./results

# 静默模式（不输出中间过程）
facg data.dat -q

# 强制使用纯 CPU 模式运行
facg data.dat --cpu

# 在当前目录下生成默认的 facg.conf 配置文件
facg --gen-config

# 在当前目录生成用于测速对比的基准测试数据集
facg --testdata
```

```
# 查看所有可用命令行选项
facg --help
```

Python API 接口调用

```
from facg import FACGConfig, run_analysis

cfg = FACGConfig(
    input_file="data.dat",
    sig_limit=5.0,
    oversampling=20.0,
    freq_high=50.0,        # 扫描频率上限 (如 d⁻¹)
    output_dir="./output", # 自定义输出路径
)
results = run_analysis(cfg)

for r in results:
    print(f"  freq = {r['freq']:12.9f}  "
          f"sig = {r['sig']:8.2f}  "
          f"amp = {r['amp']:12.9f}  "
          f"phase = {r['phase']:8.4f}  "
          f"rms = {r['rms']:12.9f}  "
          f"csig = {r['csig']:8.2f}")
```

输入格式

FACG 可直接读取 **任意由空白符分隔的文本文件** (无特定的表头或命名要求，甚至可以直接解析简单的 csv、excel 和 fits 表格)。

要求	说明
分隔符	空格或制表符等
表头	无需表头（即便有非数字开头的行也可被内部逻辑跳过）
列数	至少包含两列纯数字
第 0 列	时间戳 (可通过 `--time-col` 指定其他列)
第 1 列	观测值 / 流量 / 星等 (可通过 `--data-col` 指定其他列)

示例文件内容:

```
0.00000000 1.0022292321
0.02097133 1.0026197342
0.03765824 0.9967974816
0.06138463 0.9996638547
...
```

输出文件

所有计算结果将默认保存在与输入文件同目录的 `<原始文件名>/` 专属文件夹下，支持通过 `--output-dir` 覆盖。

最终结果汇总表 — `<stem>.dat`

保存被成功提取出的所有正弦分量。

列名	物理意义
`freq`	探测到的频率
`sig`	SigSpec 显著性
`amplitude`	该分量的半振幅
`phase`	相位角 (弧度)
`rms`	扣除迄今所有分量后的残差 RMS (均方根误差)
`csig`	SigSpec 累计显著性概率

过程文件

文件名格式	内容与意义	控制开关
`<stem>_spectrum_NNNN.dat`	第 N 次迭代的主频及残差频谱扫描阵列	`--no-spectrum` 禁用
`<stem>_residuals_NNNN.dat`	第 N 次迭代扣除模型后的时域残差	`--no-residuals` 禁用
`<stem>_spectrum_final.dat`	全流程结束后的最终残差频谱	始终输出
`<stem>_residuals_final.dat`	全流程结束后的最终时域残差	始终输出
`phase_NNNN_f*.dat`	指定被探测频率的折叠相位图	`--phase-diagrams` 开启

配置参数

参数均可通过命令行标志、`facg.conf` 文件或 Python 数据类传入。

配置文件管理

在终端执行 `facg --gen-config` 可在当前目录生成包含详细注释的 `facg.conf` 文件。运行时程序会优先读取此配置；当然，命令行中显式指定的参数标志仍具有最高优先级。

频率网格设置

属性名	CLI 命令	默认值	描述
`freq_low`	`--freq-low`	`1/T` 瑞利分辨率	频率扫描的下限
`freq_high`	`--freq-high`	`0.5/Δt` 奈奎斯特极限	频率扫描的上限
`nyquist_coeff`	`--nyquist-coeff`	0.5	奈奎斯特系数比例
`oversampling`	`--oversampling`	20.0	超采样系数 (决定了扫描粒度)

停止条件

属性名	CLI 命令	默认值	描述
`sig_limit`	`--sig-limit`	5.0	若当前残差频谱最高显著性跌破此阈值则终止分析
`csig_limit`	`--csig-limit`	0.0 (关闭)	按累积显著性设置的停止下限
`max_iter`	`--max-iter`	999	保护性设置的最大预白化迭代次数

GPU 加速策略与性能

FACG 致力于在保证天文级别计算精度的前提下，将密集型循环转移到显卡计算单元处理，主要包含三大加速域：

1. 全频谱显著性扫描 (DFT 加速)

SigSpec 不采用传统的 FFT 算法以确保对非均匀时间采样的精确性。针对包含 M 个试探频率和 N 个时间点的数据，FACG 内部构建 (M × N) 的特征矩阵积直接投递至 GPU 完成并行运算。 **同等数据集下，与纯 Python/CPU 对比，此步骤通常可获得 10–50 倍加速比。**

2. 多重正弦分量全局优化 (雅可比矩阵加速)

在传统方法中，每次探测到新的频率后，利用 SciPy `least_squares` 进行 Levenberg-Marquardt 联合拟合。若采用有限差分法估计导数，对 K 个已有频率的试探次数高达 `6K+1`，极大地拖慢了计算进度。 FACG 在推导目标函数： `y(t) = Σₖ Aₖ · sin(2π fₖ t + φₖ)` 的基础上实现了**解析型偏导数 (Analytic Jacobian)**，并在 GPU 中一步计算后整体回传给 CPU 端进行下降步迭代，彻底突破了大规模信号拟合的瓶颈。

3. 频率区间精化 (优化的纯 CPU 子程序)

与 SigSpec 一致，探测到最高峰后需要进一步利用二分法在局部极小区间逼近真实频率。因评估点过少，GPU 传输的 I/O 成本反而会带来负收益，因此此部分采用重写的高并发 NumPy 点积 (`_sig_single`) 进行计算。

算法流水线解析

FACG 的分析链路严格遵循如下经典的迭代预白化（Iterative Prewhitening）思路：

```
┌──────────────────────────────────────────────┐
│  1. 载入时序数据并去除均值（Zero-mean）     │
│  2. 构建探测频率网格列                          │
│                                                      │
│  ┌─── 预白化级联主循环 ──────────────┐   │
│  │                                                │ │
│  │  3. 计算当前残差的频谱显著性（基于 GPU）│ │
│  │  4. 定位最高显著性频率及其初始参数      │ │
│  │  5. 二分法局部极值搜索锁定真实频率      │ │
│  │  6. 迄今所有频率/振幅/相位全局重优化    │ │
│  │  7. 在原始数据中扣除优化后模型生成新残差│ │
│  │  8. 按需落盘中间临时数据与绘图            │ │
│  │                                                │ │
│  │  如果 sig < 阈值，或达到 max_iter 则退出│ │
│  └────────────────────────┘ │
│                                                      │
│                                                      │
│  9. 输出最终频域分析结果汇总表与残差文件       │
│ 10. 打印耗时总计日志                              │
└──────────────────────────────┘
```

与 SigSpec (C语言原版) 的对比

特性对比	SigSpec (原版)	FACG (本项目)
开发语言	C 语言	Python 3
GPU 并行加速	无	支持 (CUDA / Mac Metal)
显著性计算公式	Reegen 2007 论文版	完全一致
频率精化 (二分逼近)	`SigSpec_MaxSig` 函数	采用相同算法
累计显著性计算	`SigSpec_CSig` 逻辑	采用相同算法
文件读取限制	严格的命名要求 `<项目名>/`	无任何文件命名束缚
跨平台安装难度	需要配置复杂的 Makefile 编译	`pip install` 一键搞定
批量运算/分析	只能通过复杂的 `.ini` 组装	支持 CLI 原生通配符 `facg *.dat`

许可证

MIT License